

Learn Low-Level Design in 25 Days: Day 23— Trade-offs: Performance vs Readability in LLD

1. The Catch: Every Developer’s Dilemma

TechPreneur

Published November 5, 2025

Free: No

Read on Freedium · original

Contents

1. The Catch: Every Developer's Dilemma 0

2. Core Concepts: Why This Trade-off Exists 0

The Rule of Thumb 0

A Simple Example 0

Real-World Analogy 0

How to Think Like a Designer 0

ASCII Visual: The Balance 0

Case Study: Designing an In-Memory Cache 0

3. Try It Yourself 0

4. Key Takeaways 0

5. What's Next 0

1. The Catch: Every Developer's Dilemma

You've just finished optimizing your code. It's lightning fast. Every query runs in milliseconds. You proudly hit deploy — and then your teammate opens the file. They stare. They scroll. They sigh. "Who wrote this?"

If you've ever had to choose between clean, readable code and squeezing out a few extra milliseconds of performance, you've met one of the oldest trade-offs in software engineering.

Welcome to **Day 23 of "Learn Low-Level Design in 25 Days"** — where we explore **Performance vs Readability**, one of the most subtle yet critical decisions every engineer faces.



By the end of today's lesson, you'll know:

- How to think about these trade-offs in low-level design.
- When to optimize for performance, and when to keep code clean.
- How great engineers (and interviewers) expect you to reason about both.

2. Core Concepts: Why This Trade-off Exists

Let's start with the tension itself.

Readable code is like well-organized city streets — clear names, consistent layouts, easy to navigate. **Optimized code**, on the other hand, is like a secret network of tunnels — fast, but hard to find your way around.

The Rule of Thumb

> "Make it work. Make it right. Then make it fast." — Kent Beck

Readable code gets you "right." Optimized code gets you "fast."

But doing both — and knowing when to prioritize each — is what distinguishes a mature designer from a code sprinter.

A Simple Example

Let's say we're designing a class for finding the top 3 trending posts in a social media app.

Readable Version:

```
class PostAnalyzer {
    public List<Post> getTopTrendingPosts(List<Post> posts) {
        posts.sort((a, b) -> b.getLikes() - a.getLikes());
        return posts.subList(0, 3);
    }
}
```

This is clean, simple, and understandable at a glance. But it sorts all posts — even if we only need the top three.

Optimized Version:

```
class PostAnalyzer {
    public List<Post> getTopTrendingPosts(List<Post> posts) {
        PriorityQueue<Post> topPosts = new PriorityQueue<>(
            Comparator.comparingInt(Post::getLikes)
        );
        for (Post post : posts) {
            topPosts.add(post);
            if (topPosts.size() > 3) topPosts.poll();
        }
        return new ArrayList<>(topPosts);
    }
}
```

This version is faster — O(n log k) vs O(n log n) — but also more complex.

So, which one's better? **It depends.**

If this code runs once a day in a background job, readability wins. If it runs for every user every second, optimization matters.

Real-World Analogy

Think of **Uber's surge pricing system**.

- The first version may use a simple loop — easy to read, easy to debug.
- The production version likely involves in-memory caching, sharded databases, and heuristics that are far from "readable" at first glance.

At scale, readability often gives way to efficiency — but only after correctness and clarity are established.

How to Think Like a Designer

When making design choices, ask:

Frequency: How often is this code executed?

- Rarely → prioritize readability
- Millions of times per second → prioritize performance

Bottleneck: Is this on a critical path (e.g., request-response loop)?

- Yes → performance matters more
- No → clarity wins

Team Context: Will others need to modify or debug this often?

- Yes → readability is key

- No → controlled optimization is fine

Stage: Are we in early design or scaling mode?

- Early → clean, testable design
- Scaling → smart optimization, backed by profiling

ASCII Visual: The Balance

```
Performance <---------------------------> Readability
        |                                    |
   Tight loops,                          Clear naming,
   caching, pooling                      simple classes
        |                                    |
   Critical paths                     Business logic layer
```

Your job in LLD is to know where on this spectrum your module sits.

Case Study: Designing an In-Memory Cache

Let's revisit something we built earlier — a **Cache System (Day 17)**.

A cache is a perfect example of performance vs readability trade-offs:

- **Readable version:** Use a `HashMap` + basic eviction.
- **Optimized version:** Add LRU logic, object pooling, and asynchronous write-through.

Both are correct. The context — number of reads, latency goals, memory limits — decides which is right.

3. Try It Yourself

Challenge 1: Refactor one of your recent projects. Identify a function that could be optimized for performance — but at the cost of readability. How would you explain your trade-off in an interview?

Challenge 2: Design a method `getTop5Products()` for an e-commerce app. Write two versions:

- A clean, readable one.
- An optimized one (e.g., using heaps or caching). Compare their complexity and readability.

4. Key Takeaways

- **Readable code scales in teams; performant code scales in production.**
- **Never optimize blindly.** Profile before you rewrite.
- **Trade-offs are design decisions, not mistakes.**
- **Interviews test your reasoning, not just your syntax.**
- **Strive for clarity first — optimize only where it matters.**

5. What's Next

Tomorrow, we'll apply everything we've learned to a **real-world case study: Designing a Movie Ticket Booking System** — bringing together concurrency, data consistency, and trade-offs in a single cohesive design.

You don't want to miss that one.

Follow the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)

Save it, follow along, and keep building better systems — one day at a time.